

EE272 Homework 5 CNN HLS Accelerator Part 2 Report

Student Name 1: Berk Gokmen
Student Name 2: Edison Altamirano

Please answer the questions below in the space provided and include the code/reports indicated in your submission.

Passing Tests

1. Please paste a screenshot of passing C and RTL tests for all ResNet-18 layers. (5 points)

```
[eraltam@iron-05 dnn-accelerator-hls-optimized]$ python3 autograder.py rtl_test -l all
Tests being run: ['test_rtl_test']
Running rtl_test with layer params: ./layers/resnet_conv1_params.json
Test passed!

Running rtl_test with layer params: ./layers/resnet_conv2_x_params.json
Test passed!

Running rtl_test with layer params: ./layers/resnet_conv3_1_params.json
Test passed!

Running rtl_test with layer params: ./layers/resnet_conv3_x_params.json
Test passed!

Running rtl_test with layer params: ./layers/resnet_conv4_1_params.json
Test passed!

Running rtl_test with layer params: ./layers/resnet_conv4_x_params.json
Test passed!

Running rtl_test with layer params: ./layers/resnet_conv5_1_params.json
Test passed!

Running rtl_test with layer params: ./layers/resnet_conv5_x_params.json
Test passed!

Tests passed: 8
Tests run: 8
elapsed time: 3215.891570329666
```

Passing RTL tests

```
[eraltam@iron-05 dnn-accelerator-hls-optimized]$ python3 autograder.py c_test -l all
Tests being run: ['test_c_test']
Running c_test with layer params: ./layers/resnet_conv1_params.json
Test passed!

Running c_test with layer params: ./layers/resnet_conv2_x_params.json
Test passed!

Running c_test with layer params: ./layers/resnet_conv3_1_params.json
Test passed!

Running c_test with layer params: ./layers/resnet_conv3_x_params.json
Test passed!

Running c_test with layer params: ./layers/resnet_conv4_1_params.json
Test passed!

Running c_test with layer params: ./layers/resnet_conv4_x_params.json
Test passed!

Running c_test with layer params: ./layers/resnet_conv5_1_params.json
Test passed!

Running c_test with layer params: ./layers/resnet_conv5_x_params.json
Test passed!

Tests passed: 8
Tests run: 8
elapsed time: 396.7727611064911
```

Passing C tests

Design Point Capture

2. Please fill in the numbers in the following table (5 points)

Layer	Ideal	Analytical Numbers*		Unoptimized Verilog Design**		Current HLS Design***	
	Cycles	Cycles	Utilization	Cycles	Utilization	Cycles	Utilization
conv1	460992	2474767	0.1875	2475377	0.186	2855004	0.162
conv2_x	451584	451584	1.000	461165	0.979	530264	0.852
conv3_1	225792	315648	0.715	320333	0.705	373048	0.605
conv3_x	451584	589824	0.766	595517	0.758	742693	0.608
conv4_1	225792	305280	0.740	313437	0.720	376648	0.599
conv4_x	451584	589824	0.766	599997	0.753	747301	0.604
conv5_1	225792	300096	0.752	315389	0.716	383848	0.588
conv5_x	451584	589824	0.766	609149	0.741	756517	0.597

* This number considers the I/O bandwidth constraint. In other words, it takes the I/O streaming cycles into account if the layer is not compute-bound

** This is the number from homework 3 RTL implementation

*** This is your HLS design point after implementing loop pipelining, unrolling, and constant loop bound

3. Attach a waveform of your HLS simulation with signals that show the correct double buffer behavior. If you had to go through the double buffer fix, please also attach the waveform before your fix and briefly describe what you saw. (10 points)



This screenshot pulled the signals mem_cns_re_shi0, mem_cns_re_shi1, mem_cns_we_shi0, and mem_cns_we_shi1 from the WeightDoubleBuffer RTL. re signals indicate read enable signals, while we signals indicate write enable signals. Looking at the waveform, read enable signal for bank 0 is clearly high while the write enable signal for bank 1 is high as well, indicating that the double buffer behavior is working correctly. We used the solution code to start with, so we did not have to do any fixes for the double buffer.

Inner Loop Optimization

4. Given a 16x16 systolic array, calculate the ideal cycles needed to complete one output tile. Please compute it using the default tiling we provided in the handout. (5 points)
5. Explain where the extra cycles in your HLS design are coming from. (5 points)
6. Determine the performance bound for each layer in your HLS design analytically. (10 points)

Layer	Ideal Cycles for Computing One Output Tile		HLS Performance Bound
	Verilog	HLS	
conv1	9604	11123	2847488
conv2_x	7056	8172	523008
conv3_1	1764	2880	368640
conv3_x	3528	5760	737280
conv4_1	3528	5760	368640
conv4_x	7056	11520	737280
conv5_1	7056	11520	368640
conv5_x	14112	23040	737280

Ideal cycles for Verilog Calculation:

One output tile contains $OX0 \times OY0$ output pixels.

Each output pixel requires $IC0 \times FX \times FY \times IC1 \times OC0$ MACs.

Each output tile requires $IC0 \times FX \times FY \times IC1 \times OC0 \times OX0 \times OY0$ MACs

Ideal cycles is: $IC0 \times FX \times FY \times IC1 \times OC0 \times OX0 \times OY0 / (IC0 \times OC0) = FX \times FY \times IC1 \times OX0 \times OY0$.

Ideal Cycles for HLS Calculation:

Each output tile requires going through the loop $IC1 \times FX \times FY$ times

Each loop requires loading $OX0 \times OY0$ input vectors.

Systolic array requires $IC0 + OC0 - 1$ cycles to compute the computation.

Ideal cycles is: $IC1 \times FX \times FY \times (OX0 \times OY0 + IC0 + OC0 - 1)$

Extra cycles: The extra cycles in the HLS version comes from the fact that the systolic array restarts $IC1 \times FX \times FY$ times. This means, we need to wait $IC0 + OC0 - 1$ for the systolic array to fill up again $IC1 \times FX \times FY$ times. This is a major inefficiency!

Performance Bound:

Compute: ideal cycles $\times (OC1 \times OX1 \times OY1)$

Input read: $IX0 \times IY0 \times IC1 \times IC0 \times (OX1 \times OY1) / 4$

Weight read: $FY \times FX \times IC1 \times IC0 \times OC0 \times (OC1 \times OX1 \times OY1) / 4$

Output write: $(OX0 \times OY0 \times OC0) \times (OC1 \times OX1 \times OY1)$

Bound = max(compute, input read, weight read, output write)

Evaluating Accelerator Performance

7. What optimizations did you make and how did they improve utilization or reduce the size of your design? (10 points)

C++ Pragma-Based Parallelism

InputDoubleBuffer.h

```
#pragma hls_pipeline_init_interval 1
TILES: for (int t = 0; t < OX1_MAX * OY1_MAX; t++) {
```

This loop controls the streaming rate of how tiles are recorded. Pipelining here helps us sustain throughput while it repeatedly fills the tile buffer. If it's not there: Catapult may schedule one tile at a time with extra control overhead between iterations, creating bubbles and reducing throughput.

```
PackedInt<INPUT_PRECISION, 4> packet = din.read();
#pragma hls_unroll yes
for (int k = 0; k < 4; k++) {
    memCol.value[j+k] = packet.value[k];
}
```

This loop is always exactly 4 iterations and is purely “lane unpacking”. Unrolling removes loop control and turns it into simple parallel assignments, so unpacking doesn't cost cycles. Otherwise, The tool can implement a tiny sequential loop/FSM to unpack, adding unnecessary cycles or control logic that can slow down the writer.

```
#pragma hls_pipeline_init_interval 1
OC1: for (int oc1 = 0; oc1 < OC1_MAX; oc1++) {
    IC1: for (int ic1 = 0; ic1 < IC1_MAX; ic1++) {
        FY: for (int fy = 0; fy < FY_MAX; fy++) {
            FX: for (int fx = 0; fx < FX_MAX; fx++) {
                OY0: for (int oy0 = 0; oy0 < OY0_MAX; oy0++) {
                    OX0: for (int ox0 = 0; ox0 < OX0_MAX; ox0++) {
                        uint_16 address =
                            params.STRIDE * ox0 + fx +
                            (params.STRIDE * oy0 + fy) * IX0 +
                            IY0 * IX0 * ic1;
                        dout.write(tmp.data[address]);
```

OC1 is reuse-driven and sits near the top of the reader's nested loops. Pipelining it helps keep a steady stream of outputs when iterating OC1 reuse patterns. If not present, we can get scheduling gaps between OC1 iterations, meaning the downstream core receives data less regularly.

WeightDoubleBuffer.h

```
#pragma hls_pipeline_init_interval 1
TILES: for (int t = 0; t < OX1_MAX * OY1_MAX * OC1_MAX; t++) {
    chanStruct<PackedInt<WEIGHT_PRECISION, OC0>,size> tmp;
    TILE: for (int i = 0; i < FX_MAX * FY_MAX * IC1_MAX * IC0_MAX; i++) {
```

Weight tiles are produced repeatedly; pipelining the tile loop helps the writer keep up and avoids starvation downstream. If it's not there: The core may wait on weights (backpressure), lowering PE utilization.

```
PackedInt<WEIGHT_PRECISION, OC0> memRow;
for (int j = 0; j < OC0_MAX; j=j+4) {
    PackedInt<WEIGHT_PRECISION, 4> packet = din.read();
    #pragma hls_unroll yes
    for (int k = 0; k < 4; k++) {
        memRow.value[j+k] = packet.value[k];
    }
    if (j == OC0 - 4) break;
}
```

Exactly the same rationale as inputs: fixed size (4), pure wiring/unpack, so unroll is cheap and prevents extra control cycles. Otherwise, unpacking can become a small but real bottleneck and add bubbles when weights are produced.

```
#pragma hls_pipeline_init_interval 1
TILES: for (int t = 0; t < OX1_MAX * OY1_MAX * OC1_MAX; t++) {
    chanStruct<PackedInt<WEIGHT_PRECISION, OC0>,size> tmp;
    tmp = din.read();
    TILE: for (int i = 0; i < FX_MAX * FY_MAX * IC1_MAX * IC0_MAX; i++) {
        dout.write(tmp.data[i]);
        if (i == tileSize - 1) break;
    }
    if (t == params.OX1 * params.OY1 * params.OC1 - 1) break;
}
```

The reader is responsible for streaming weights at a consistent rate; pipelining at tile level helps reduce stalls between tiles. If not present, weight delivery becomes slow, and the systolic array can stall.

SystolicArrayCore.h

```
#pragma hls_pipeline_init_interval 1
void CCS_BLOCK(run){
    ac_channel<PackedInt<INPUT_PRECISION, IC0> > &input,
    ac_channel<PackedInt<WEIGHT_PRECISION, OC0> > &weight,
    ac_channel<PackedInt<OUTPUT_PRECISION, OC0> > &output,
    ac_channel<Params> &paramsIn,
    ac_channel<LoopIndices> &loopIndicesIn)
{
```

run() is the throughput-critical region. Pipelining this is what keeps the systolic datapath continuously active.

```
PackedInt<WEIGHT_PRECISION, OC0> w_row = weight.read();
#pragma hls_unroll yes
for(int j = 0; j < OC0_MAX; j++){
    weight_reg[step][j] = w_row.value[j];
    if (j == OC0 - 1) break;
}
```

This fills one PE-row of weights (lane width). If done sequentially, the array gets weights late and it can create bubbles. Otherwise, weight loading can take OC0 cycles (or add control), delaying compute and reducing utilization.

```
#pragma hls_unroll yes
LABEL(INIT_IN) for(int i = 0; i < IC0_MAX; ++i) {
    input_reg[i][0] = input_buf.value[i];
    if (i == IC0 - 1) break;
}
```

Boundary injection for a systolic array should occur “at once” across IC0 lanes, not over multiple cycles. If not present, Inputs get loaded sequentially, which can stall the array or force extra buffering.

```
#pragma hls_unroll yes
for(int j = 0; j < OC0_MAX; j++){
    psum_buf.value[j].template set_val<AC_VAL_0>();
    if (j == OC0 - 1) break;
}
```

This is lane-wise initialization. If rolled, it spend OC0 cycles just clearing psums so it has wasted cycles. Not putting in there would mean wasting cycles before compute starts (startup overhead), hurting performance.

```
#pragma hls_unroll yes
for(int j = 0; j < OC0_MAX; j++){
    psum_buf.value[j] = accumulation_buffer[step][j];
    if (j == OC0 - 1) break;
}
```

Psum fetch must match lane width to avoid stalling compute. By doing that, Psum fetch doesn't become serialized and stalls the compute pipeline.

```
#pragma hls_unroll yes
LABEL(INIT_OUT) for(int j = 0; j < OC0_MAX; ++j)
    psum_reg[0][j] = output_buf.value[j];
    if (j == OC0 - 1) break;
}
```

Boundary psums must be available to the first PE row immediately. Otherwise, boundary psums fill slowly and delay valid compute.

```
#pragma hls_unroll yes
LABEL(COL) for (int j=0; j < OC0_MAX; ++j) {
    #pragma hls_unroll yes
    LABEL(ROW) for (int i=0; i < IC0_MAX; ++i) {
        pe[i][j].run(input_reg[i][j], psum_reg[i][j], weight_reg[i][j], input_reg2[i][j], psum_reg2[i][j]);
        if (i == IC0 - 1) break;
    }
    if (j == OC0 - 1) break;
}
```

These loops define the physical PE array. Unrolling them prevents time-multiplexing and ensures all PEs run in parallel in the same cycle. We do that to prevent Catapult run them sequentially, massively lowering utilization and increasing cycles.

```
#pragma hls_unroll yes
for(int i = 0; i < OC0_MAX; i++){
    accumulation_buffer[step-(IC0+OC0-1)][i] = output_row.value[i];
    if (i == OC0 - 1) break;
}
if (loopIndices.ic1_idx==params.IC1-1 && loopIndices.fx_idx == params.FX-1 && loopIndices.fy_idx == params.FY-1) {
    output.write(output_row);
}
```

Lane-parallel writeback avoids stretching a single output row across OC0 cycles. By doing that Writeback cannot serialize and throttle the pipeline.


```

#pragma hls_unroll yes
for(int j = 0; j < OC0_MAX; j++){
    #pragma hls_unroll yes
    for(int i = 0; i < IC0_MAX; i++){
        input_reg[i][j+1] = input_reg2[i][j];
        psum_reg[i+1][j] = psum_reg2[i][j];
        if (i == IC0 - 1) break;
    }
    if (j == OC0 - 1) break;
}

```

This is the per-cycle systolic advance a step. It must happen in parallel; rolling it creates control overhead.

SystolicArray.h

```

#pragma hls_design interface
#pragma hls_pipeline_init_interval 1
void run(ac_channel<PackedInt<INPUT_PRECISION, IC0> > &input,
        ac_channel<PackedInt<WEIGHT_PRECISION, OC0> > &weight,
        ac_channel<PackedInt<OUTPUT_PRECISION, OC0> > &output,
        ac_channel<Params> &paramsIn)
{
    systolicArrayLooper.run(paramsIn, paramsChannel, loopIndicesChannel);
    systolicArrayCore.run(input, weight, output, paramsChannel, loopIndicesChannel);
}

```

It helps maintain a steady streaming schedule at the wrapper/top integration level. If it's not there, the wrapper can introduce scheduling gaps even if the core is optimized.

TCL-Based Scheduling and Resource Optimizations

SystolicArrayCore.tcl/Conv.tcl:

In the TCL files, We optimized memory organization (interleaving and block size) to reduce memory conflicts, mapped critical per-cycle arrays to registers for faster access, removed conservative memory dependencies to allow tighter scheduling, and set small FIFO depths to reduce backpressure. These changes help maintain II=1 and improve utilization while controlling area growth.

Loop Bounds

To provide fixed trip counts to the HLS tool, We specifically established static loop bounds (such as IC0_MAX, OC0_MAX, and OX1_MAX) for all for loops in addition to using unroll and pipeline pragmas. This enables Catapult to safely implement extreme optimizations like pipelining and full lane-level unrolling and to properly evaluate the iteration space. Without compile-time loop bounds, the scheduler would have to be conservative, potentially limiting unrolling, increasing initiation interval, and inserting additional control logic to guard against uncertain iteration counts. These choices, along with TCL-level memory interleaving, register

mapping, and false-dependency removal, as well as pragma-based parallelization in the systolic core and streaming buffers, guarantee that the inner compute loops run with consistent throughput while avoiding needless control overhead or artificial stalls.

Inner Loop Optimization

Inner loop optimization streams across the work that is needed to compute the output tile, and this streaming, prevents the array from restarting $IC1 \times FX \times FY$ times as described earlier in the report. Every restart causes an overhead of $IC0+OC0-1$, which is only paid for once per output tile.

8. In the table below, list the number of cycles your final optimized accelerator takes to run each layer and the MAC utilization. Please highlight the layers where you meet the target performance or Verilog performance. (50 points)

Layer	Target Performance		Unoptimized Verilog Design		Final HLS Design	
	Cycles	Utilization	Cycles	Utilization	Cycles	Utilization
conv1	2910000	0.158	2475377	0.186	2658654	0.173
conv2_x	540000	0.836	461165	0.979	494490	0.913
conv3_1	380000	0.594	320333	0.705	301498	0.749
conv3_x	755000	0.598	595517	0.758	597463	0.756
conv4_1	384000	0.588	313437	0.720	304010	0.743
conv4_x	760000	0.594	599997	0.753	600983	0.751
conv5_1	391000	0.577	315389	0.716	310666	0.727
conv5_x	770000	0.586	609149	0.741	609655	0.741